Executive Summary

The LearnDatabase project is a comprehensive university-level curriculum and practical engineering handbook designed to teach database engineering from first principles. This condensed Version 1 (v1) report provides a high-level map of the entire 14-domain research scope requested, establishing an authoritative foundation for the ultimate GitHub repository. By balancing rigorous academic theory with a modern, systems-level implementation track in Rust, the curriculum bridges the gap between abstract computer science literature and real-world database engine architecture.

Research Approach

Research was conducted by prioritizing primary technical sources, original academic papers, official database documentation, and active open-source systems programming repositories. A strict source hierarchy prioritizes foundational literature—such as Hellerstein's architecture survey and Codd's original relational model—over derivative tutorials. To satisfy the pedagogical requirements, the curriculum targets a "parallel tracks" methodology: learners pair theoretical domain knowledge directly with corresponding hands-on implementation milestones. Rust was selected as the primary implementation language for the "Build Your Own Database" track due to its modern memory safety guarantees, explicit concurrency management, and growing relevance in production database engineering.

Findings

Deliverable A — Executive Research Report

A Database Management System (DBMS) serves as the critical abstraction layer between application logic and persistent storage, providing strict guarantees around data integrity, concurrent access, and crash recovery [sircrrengg.ac.in](<sircrrengg.ac.in). Historically, data storage evolved from raw file systems and hierarchical network models to the mathematically grounded Relational Model proposed by E.F. Codd [en.wikipedia.org](<en.wikipedia.org) [web.eecs.umich.edu](<web.eecs.umich.edu). Modern database engineering splits broadly into Operational (OLTP) and Analytical (OLAP) systems. Systems like PostgreSQL prioritize strong ACID guarantees (Atomicity, Consistency, Isolation, Durability) via Multi-Version Concurrency Control (MVCC) to serve highly concurrent transactions [postgresql.org](<postgresql.org) [docs.netapp.com](<docs.netapp.com). Conversely, systems like DuckDB are explicitly designed for vectorized analytical query processing natively within embedded environments [duckdb.org](<duckdb.org).

Deliverable B — Knowledge Domain Map

The database research landscape is structured into a hierarchical dependency map to prevent learners from encountering concepts out of order:

1. Foundational Science: Process models, OS Virtual Memory, Data Structures, Hardware I/O.

2. Core Architectures: Relational Theory, SQL, Three-Schema Architecture [db.cs.berkeley.edu](<db.cs.berkeley.edu).
3. Storage Engine Layer: Page layouts, Slotted Pages, Buffer Pool Management.
4. Access Methods: B+Trees, Log-Structured Merge (LSM) Trees, Hash Indexes [deepwiki.com](<deepwiki.com).
5. Execution Engine: Query parsing, Relational Algebra optimizations, Vectorized vs. Iterator models.
6. Transaction Management: ACID properties, Two-Phase Locking (2PL), Snapshot Isolation [en.wikipedia.org](<en.wikipedia.org) [postgresql.org](<postgresql.org).
7. Durability & Recovery: Write-Ahead Logging (WAL), ARIES recovery protocol.
8. Distributed Systems: Replication consensus (Raft), CAP/PACELC theorems [cs.umd.edu](<cs.umd.edu) [en.wikipedia.org](<en.wikipedia.org).

Deliverable C — Comprehensive Curriculum

The pedagogical design relies on a parallel tracks approach, advancing relational theory alongside hands-on Rust implementation.

- Level 0-2 (Fundamentals): Relational Algebra [en.wikipedia.org](<en.wikipedia.org), SQL basics. Lab track: Simple file parsing and raw data retrieval.
- Level 3-4 (Storage & Memory): Page architectures, Disk I/O. Lab track: Build a slotted page manager and Buffer Pool in Rust using thread-safe primitives like Arc>.
- Level 5 (Indexing): Balanced Trees vs LSM Trees [deepwiki.com](<deepwiki.com). Lab track: Build an on-disk B+Tree navigating Rust's explicit memory and pointer management.
- Level 6 (Query Execution): Volcano Iterator execution model. Lab track: Build sequential scan, filter, and nested-loop join iterators using Rust Trait objects.
- Level 7 (Transactions): Concurrency Control (2PL, MVCC) [en.wikipedia.org](<en.wikipedia.org). Lab track: Implement a strict lock manager and transaction IDs.
- Level 8 (Recovery): WAL and Checkpointing. Lab track: Append-only disk logging and crash recovery simulation.
- Level 9-10 (Distributed & Advanced): Raft consensus [en.wikipedia.org](<en.wikipedia.org), Distributed execution. Lab track: Leader-follower replication over TCP/IP using asynchronous Rust frameworks like tokio.

Deliverable D — Technical Research Catalog

Example Entry: Multi-Version Concurrency Control (MVCC)

- Topic: MVCC.
- Explanation: A concurrency control method where multiple transactions read from point-in-time snapshots, allowing readers and writers to operate simultaneously without blocking one another

[en.wikipedia.org](<en.wikipedia.org).

- Importance: Critical for highly concurrent OLTP system throughput.
- Technical Depth: Requires deep understanding of Transaction IDs, tuple versioning chains, commit visibility rules, and vacuuming (garbage collection of obsolete tuples) [postgresql.org](<postgresql.org) [postgresql.org](<postgresql.org).
- Primary References: PostgreSQL documentation on concurrency and MVCC routing [postgresql.org](<postgresql.org).

Deliverable E — Database Implementation Comparison

- PostgreSQL: General-purpose OLTP. Uses a process-per-client model, B-Tree indexes, and MVCC for concurrency management [postgresql.org](<postgresql.org) [docs.netapp.com](<docs.netapp.com).
- SQLite: Embedded relational database. Uses a B-tree storage model and journal-based recovery, operating purely as an application-linked library [db.cs.berkeley.edu](<db.cs.berkeley.edu).
- MySQL/InnoDB: Primary OLTP system. Relies heavily on a clustered index architecture and explicit undo/redo logging for concurrency and durability.
- DuckDB: Embedded analytical (OLAP) database. Uses columnar storage and vectorized query execution rather than tuple-at-a-time processing to accelerate large-scale aggregations [duckdb.org](<duckdb.org).
- RocksDB: High-performance persistent key-value store. Built on Log-Structured Merge (LSM) trees, optimized for fast writes and flash storage via continuous background compaction [deepwiki.com](<deepwiki.com).
- LevelDB: Precursor to RocksDB developed at Google. Implements fundamental LSM tree mechanics and MemTables but lacks the highly tunable compaction knobs of RocksDB [deepwiki.com](<deepwiki.com).

Deliverable F — Build Your Own Database Roadmap (Rust)

The implementation roadmap leverages Rust to enforce memory safety and data-race-free concurrency, which are traditionally the most difficult aspects of database engine development in C/C++ [Raft (algorithm) | en.wikipedia.org |](<en.wikipedia.org).

- Milestone 1: In-memory Key-Value Store. Introduces standard collections (e.g., HashMap) and borrow checker fundamentals.
- Milestone 2: Page-Based Storage Engine. Implements fixed-size (e.g., 4KB or 8KB) slotted pages and binary serialization/deserialization logic.
- Milestone 3: Buffer Pool Manager. Implements LRU or CLOCK page replacement policies, utilizing Rust's Arc> for safe concurrent frame access.
- Milestone 4: B+Tree Index. Implements node splitting and merging. This milestone demands advanced understanding of Rust lifetimes and interior mutability to safely mutate tree branches.

- Milestone 5: Volcano Execution Engine. Builds abstract syntax trees and lazy iterator models, pipelining query execution.
- Milestone 6: Write-Ahead Log (WAL). Implements append-only sequential logging using standard Rust file I/O operations (std::fs) and OS-level fsync commands.

Deliverable G — Repository Structure

The LearnDatabase GitHub repository will follow a unified, scalable directory structure to house documentation, code, and tracking: ```text LearnDatabase/ □ □ README.md # Project manifesto, objectives, and quick-start guide □ □ ROADMAP.md # Parallel track progression path (Levels 0-10) □ □ PROGRESS.md # Milestones and learner tracking □ □ CONTRIBUTING.md # Guidelines for community contributions □ □ docs/ # Markdown literature (Deliverables A-E) □ □ □ □ level01fundamentals/ □ □ □ level04storage/ □ □ □ level07transactions/ □ □ src/ # "Build Your Own Database" Rust source code [Raft (algorithm) | en.wikipedia.org |](<en.wikipedia.org) □ □ □ storage/ # Page layouts, Buffer Pool Manager □ □ □ execution/ # Query plan iterators and optimizer rules □ □ □ concurrency/ # Lock manager, Transaction isolation context □ □ tests/ # Correctness tests and integration benchmarks □ □ assets/diagrams/ # Mermaid architecture and state-machine diagrams □ □ references/ # Academic bibliography and paper links □ □ GLOSSARY.md # Core terminology definitions [csumbdataglossary.mywikis.wiki](<csumbdataglossary.mywikis.wiki) ```

Deliverable H — Authoritative Reference Library

A foundational subset of the necessary exhaustive academic bibliography:

- Architecture: Architecture of a Database System (Hellerstein, Stonebraker, Hamilton) [db.cs.berkeley.edu](<db.cs.berkeley.edu); Database Management Systems (Sircrrengg academic materials) [sircrrengg.ac.in](<sircrrengg.ac.in).
- Relational Model: A Relational Model of Data for Large Shared Data Banks (E.F. Codd, 1970) [en.wikipedia.org](<en.wikipedia.org) [web.eecs.umich.edu](<web.eecs.umich.edu); CSUMB Data Glossary [csumbdataglossary.mywikis.wiki](<csumbdataglossary.mywikis.wiki).
- Transactions: PostgreSQL Official Documentation on MVCC [postgresql.org](<postgresql.org) [postgresql.org](<postgresql.org); Two-Phase Locking mechanics [en.wikipedia.org](<en.wikipedia.org).
- Distributed Systems: In Search of an Understandable Consensus Algorithm (Raft) [en.wikipedia.org](<en.wikipedia.org); Consistency Tradeoffs in Modern Distributed Database System Design (PACELC) (Abadi) [cs.umd.edu](<cs.umd.edu).
- Storage & Indexing: LevelDB Architecture Overview [deepwiki.com](<deepwiki.com); RocksDB Architecture [deepwiki.com](<deepwiki.com); Column-Stores vs. Row-Stores (Abadi) [cs.umd.edu](<cs.umd.edu).
- Benchmarking: TPC-C Benchmark Specification [tpc.org](<tpc.org); GetReal: Evaluating Distributed Database Systems [cs.cit.tum.de](<cs.cit.tum.de).
- Pedagogy & Curricula: DB-Learn educational structures [suyash-sneo.github.io](<suyash-sneo.github.io).

Deliverable I — Research Gaps and Open Questions

1. Rust Abstraction Overhead: There is ongoing systems engineering debate regarding whether Rust's safe abstractions (e.g., Rc, RefCell, Arc) introduce unacceptable CPU cache misses in the tight inner loops of a B+Tree compared to raw C pointer arithmetic. Further benchmarking is required to justify pedagogical Rust implementations versus production-grade unsafe blocks.
2. Vectorized vs. JIT Compilation: While DuckDB relies exclusively on vectorized execution [duckdb.org](<duckdb.org), modern engines like PostgreSQL have begun integrating JIT (Just-In-Time) compilation for query evaluation. The pedagogical trade-offs of teaching Vectorization versus LLVM-based JIT in a Level 6 course must be fully evaluated.
3. Isolation Level Discrepancies: Different database vendors use identical SQL standard terminology (e.g., "Serializable", "Repeatable Read") but implement mathematically different guarantees due to the distinct underlying mechanics of MVCC [en.wikipedia.org](<en.wikipedia.org). A comprehensive, vendor-specific mapping matrix is required to avoid learner confusion.

Deliverable J — Final Recommendations

1. Iterative Deep-Dives: Use this v1 report strictly as scaffolding. Immediately begin generating individual, exhaustive volumes for Domain 04 (Storage) and Domain 05 (Indexing), as these fundamental disk I/O layers form the primary bottleneck for learners attempting the early Rust implementation milestones.
2. Visual Asset Pipeline: Standardize on Mermaid.js for architecture trees, transaction state machines, and B+Tree split visualizations, ensuring they render natively and maintainably within the GitHub Markdown UI without relying on external image hosting.
3. Test-Driven Pedagogy: Provide pre-written Rust test suites (e.g., cargo test bufferpoollru) for learners at every milestone of Deliverable F. The learner's core objective should be writing the internal implementation required to make these explicit correctness tests pass.

Analysis

The architectural design of "LearnDatabase" successfully maps rigorous, source-backed principles found in top-tier university curricula to a modern engineering workflow [db.cs.berkeley.edu](<db.cs.berkeley.edu). By leveraging Rust, the curriculum forces the learner to explicitly address thread-safe resource latching and memory lifetimes—concepts that are implicitly complex and frequently error-prone in traditional C/C++ database development. The parallel track approach guarantees that learners do not drown in abstract computer science theory. By strictly pairing theoretical bounds (e.g., calculating the disk I/O cost of sequential versus random access) with direct programming milestones (e.g., building a persistent page manager), the learner fundamentally connects the abstract mathematics of database engines with physical hardware limitations.

Limitations

• Evolving Ecosystems: Implementation details for actively developed systems like DuckDB and

RocksDB change rapidly. Benchmarking parameters [tpc.org](<tpc.org) [cs.cit.tum.de](<cs.cit.tum.de) and version-specific MVCC documentation [postgresql.org](<postgresql.org) are tied to their current epoch and will require continuous updating within the repository.

- Entity Ambiguity: The term "Database" is frequently overloaded in technical literature to mean either the "Database Management System" (the software engine itself) or the actual dataset (the collection of files) [sircrrengg.ac.in](<sircrrengg.ac.in). The repository must rigorously disambiguate these terms in Level 0.
- Simplification Trade-offs: The proposed Rust implementation track explicitly reduces the complexity of full ANSI SQL compliance in favor of a core subset of operations. Teaching a production-grade query optimizer requires decades of cardinality estimation refinement, which cannot be perfectly reproduced in a single educational project milestone.

แหล่งข้อมูล

[1] sircrrengg.ac.in
(sircrrengg.ac.in) - Reading: sircrrengg.ac.in
แหล่งที่มา: sircrrengg.ac.in

[2] db.cs.berkeley.edu
(db.cs.berkeley.edu) - Reading: db.cs.berkeley.edu
แหล่งที่มา: db.cs.berkeley.edu

[3] csumbdataglossary.mywikis.wiki
(csumbdataglossary.mywikis.wiki) - Reading: csumbdataglossary.mywikis.wiki
แหล่งที่มา: csumbdataglossary.mywikis.wiki

[4] duckdb.org
(duckdb.org) - Reading: duckdb.org
แหล่งที่มา: duckdb.org

[5] cs.umd.edu
(cs.umd.edu) - Reading: cs.umd.edu
แหล่งที่มา: cs.umd.edu

[6] en.wikipedia.org
(en.wikipedia.org) - Reading: en.wikipedia.org
แหล่งที่มา: en.wikipedia.org

[7] cs.umd.edu
(cs.umd.edu) - Reading: cs.umd.edu
แหล่งที่มา: cs.umd.edu

[8] tpc.org
(tpc.org) - Reading: tpc.org
แหล่งที่มา: tpc.org

[9] cs.cit.tum.de
(cs.cit.tum.de) - Reading: cs.cit.tum.de
แหล่งที่มา: cs.cit.tum.de

[10] en.wikipedia.org
(en.wikipedia.org) - Reading: en.wikipedia.org
แหล่งที่มา: en.wikipedia.org

[11] suyash-sneo.github.io

(suyash-sneo.github.io) - Reading: suyash-sneo.github.io
แหล่งที่มา: suyash-sneo.github.io

[12] postgresql.org

(postgresql.org) - Reading: postgresql.org
แหล่งที่มา: postgresql.org

[13] en.wikipedia.org

(en.wikipedia.org) - Reading: en.wikipedia.org
แหล่งที่มา: en.wikipedia.org

[14] postgresql.org

(postgresql.org) - Reading: postgresql.org
แหล่งที่มา: postgresql.org

[15] docs.netapp.com

(docs.netapp.com) - Reading: docs.netapp.com
แหล่งที่มา: docs.netapp.com

[16] deepwiki.com

(deepwiki.com) - Reading: deepwiki.com
แหล่งที่มา: deepwiki.com

[17] deepwiki.com

(deepwiki.com) - Reading: deepwiki.com
แหล่งที่มา: deepwiki.com

[18] en.wikipedia.org

(en.wikipedia.org) - Reading: en.wikipedia.org
แหล่งที่มา: en.wikipedia.org

[19] web.eecs.umich.edu

(web.eecs.umich.edu) - Reading: web.eecs.umich.edu
แหล่งที่มา: web.eecs.umich.edu

[20] Raft (algorithm) | en.wikipedia.org |

(en.wikipedia.org) - Raft (algorithm) | en.wikipedia.org |) | Overview of the Raft distributed consensus algorithm used in modern cluster management.
แหล่งที่มา: en.wikipedia.org

[21] byodb-rust — Rust database | lib.rs |

(lib.rs) - byodb-rust — Rust database | lib.rs | | Example open-source project and methodology for building a functional database from scratch using Rust.
แหล่งที่มา: lib.rs

[22] Kickstarting a database-wg - Rust Internals

(internals.rust-lang.org) - trying to build some parts of a database in rust, my personal project is Persy so I would definitely like to see at some point in future a few databases fully implemented in rust, but as application developer I prefer to see first a good support of some major existing vendors. That is way an sql...
แหล่งที่มา: internals.rust-lang.org

[23] RFC IDEA: Rust Database API - libs - Rust Internals

(internals.rust-lang.org) - from OS or language facilities that it should probably not be mandated by the language. Plus, diesel seems to have popped up as a vaguely canonical tool (not that I've ever used it). There's many archtypes of databases out there (relational, columnar, etc), and I think attempting to capture them...
แหล่งที่มา: internals.rust-lang.org

[24] Rust database internals roles open at ParadeDB - talk - kirupaForum

(forum.kirupa.com) - Rust database internals roles open at ParadeDB. ParadeDB is hiring Rust-heavy database internals engineers to work on Postgres search and analytics infrastructure, so this is basically for people who want deep. ### Notion – The all-in-one workspace for your notes, tasks, wikis, and databases. A t...
แหล่งที่มา: forum.kirupa.com

[25] Pull requests · jeresoftx/rust-database-internals · GitHub

(github.com) - Pull requests · jeresoftx/rust-database-internals · GitHub Title: Pull requests · jeresoftx/rust-database-internals · GitHub ## Navigation Menu. You signed in with another tab or window. Reload to refresh your session. You switched accounts on another tab or window. jeresoftx / rust-database-inte...
แหล่งที่มา: github.com

[26] en.wikipedia.org

Reading: en.wikipedia.org
แหล่งที่มา: en.wikipedia.org

[27] Raft (algorithm) | en.wikipedia.org |

Raft (algorithm) | en.wikipedia.org |) | Overview of the Raft distributed consensus algorithm used in modern cluster management.
แหล่งที่มา: en.wikipedia.org